

自适应门控事实记忆：从选择性长期存储到精确词元恢复

作者姓名

单位名称
电子邮箱

1 问题背景

从全局注意力到固定状态推理

标准自注意力允许第 t 个词元访问此前的全部位置，因此具有很强的长程依赖建模能力。对于长度为 L 、隐藏维度为 D 的序列，其训练计算量随 L^2 增长；自回归推理虽然可以复用历史的键和值，仍需保存与历史长度成正比的键值缓存 (KV 缓存) (Vaswani et al. 2017)。当对话持续增长时，每个新词元都会带来新的状态，显存占用和单步读取成本随之增加。为缓解这一问题，已有工作提出 Longformer、Memformer、Linformer、Performer、Reformer 和 Keyformer 等架构 (Beltagy et al. 2020; Wu et al. 2020; Wang et al. 2020; Choromanski et al. 2021; Kitaev et al. 2020)，分别通过滑动窗口、历史状态压缩、注意力核近似、哈希分桶和推理端缓存筛选来降低长序列开销。然而，不同工作使用的数据、模型规模和训练设置并不一致，难以判断差异究竟来自架构还是实验条件。更根本的问题是：当历史被压缩或覆盖时，有限状态究竟应该保留什么，才能避免有用信息被无关内容淹没？

滑动窗口注意力 (sliding-window attention, SWA) 的因果语言模型用固定长度的键值缓存控制推理状态。设局部注意力预算为 B ，在时间步 t ，模型只能访问少量初始汇聚词元和最近的上下文：

$$\mathcal{C}_t = \mathcal{C}_{\text{sink}} \cup \mathcal{C}_{\text{recent}}, \quad |\mathcal{C}_t| \leq B. \quad (1)$$

在每层只读取 B 个局部键值时，注意力计算量约为 $\mathcal{O}(LBD)$ ，流式键值状态约为 $\mathcal{O}(N_{\text{layer}}BD)$ ；当 B 固定时，二者都不再随完整历史无限增长。保留少量汇聚词元可以缓解窗口滑过序列开头后出现的注意力退化 (Xiao et al. 2024)。近期的大规模比较还表明，带汇聚词元的简单 SWA 在多个短、长上下文基准上可以达到或超过复杂的后训练线性注意力方法，并具有较好的速度和内存效率 (Jolicoeur-Martineau et al. 2026)。因此，评价新的记忆机制时，必须把结构简单、实现成熟的 SWA-sink 作为强基线，并同时报告质量、状态和计算成本。下文中，KV (键值) 表示由键和值组成的缓存对；为避免与长期事实槽位混淆，局部通道称为“键值缓存”，长期通道称为“事实槽位”。

不过，这些方法并未从根本上解决长期事实记忆问题。SWA 通过丢弃窗口外的逐词元表示来控制成本；它可以利用多层感受野和参数中的统计知识，却不能保证数千个词元之后仍能逐字恢复任意早期事实。即使保留少量汇聚词元，窗口外的其余内容也无法直接访问。状态

压缩模型则可能在话题切换或反复压缩后遗忘细节。另一方面，自然语言具有较高冗余性，真正需要长期保存的内容通常只占历史的一小部分：用户明确要求保存的偏好可能需要跨越很长的干扰文本，而临时说明和一次性事实不应持续占用状态。因此，长期记忆的关键不只是扩大窗口，还包括判断哪些内容值得写入、已存内容是否仍应保留，以及查询时应读取哪一条记忆。本文在局部窗口之外增设 8 个可学习的长期事实槽位，把写入、保留和读取决策交给模型学习。

选择性记忆问题的形式化

设一段对话由轮次 $x_1, \dots, x_T$ 组成，每轮可能产生事实候选集合 $\mathcal{F}_t$ 。系统维护最多 M 个长期槽位 $\mathcal{M}_t$ ，并满足

$$|\mathcal{M}_t| \leq M, \quad \mathcal{M}_{t+1} = U(\mathcal{M}_t, \mathcal{F}_t), \quad (2)$$

其中 U 同时包含拒绝、写入、合并、更新和淘汰。对后续查询 q ，读取器从 $\mathcal{M}_t$ 中选出少量槽位 $R(q, \mathcal{M}_t)$ ，语言模型再根据局部上下文与检索结果生成答案 y 。因此，端到端正确率至少受四类事件共同影响：目标事实是否被写入，是否在干扰后仍然存活，是否被读取器选中，以及检索表示能否被解码为正确的词元序列。

这种分解可以区分不同的失败模式。若存活率下降，问题在容量分配或保留；若存活率高而召回率低，问题在语义寻址；若二者均高而完全匹配率仍低，问题位于记忆融合或答案解码。本文围绕这一分解组织架构和实施，避免用单个端到端分数掩盖内部机制差异。

从语义记忆到精确词元的第二个瓶颈

固定槽位记忆能够限制状态规模，但简单的“见到事实就写入”策略会产生容量竞争。当临时事实不断进入有限槽位时，早期的重要事实可能被最近最少使用 (least recently used, LRU) 规则淘汰。内容门控可以缓解这一问题：模型学习写入概率和保留概率，只让重要事实进入长期状态。然而，长期事实问答还包含另一个容易被忽略的环节，即从检索到的连续记忆表示恢复精确答案词元。一个槽位可以在语义上表示“Bob 的偏好与蓝色有关”，却未必能使语言模型在多个颜色词中稳定地把 blue 排在第一位。

本研究将长期事实恢复分解为三个相互关联但可分别测量的阶段：选择性存储、语义检索和词级生成。实验关注两个问题：其一，内容门控的有界槽位能否在长延迟和事实型噪声下持续保存目标事实；其二，当目标槽位已经被正确检索时，显式的值片段建模能否缩小连续语义表示与离散答案词元之间的差距。我们还在

Table 1: 事实记忆与六种基础路径的机制对照。六种基础架构的长期事实指标在当前 TinyStories 语言建模协议中未测量，因此不以“不适用”伪装成零。

方案	训练时是否改变注意力或状态	长期状态形式	是否能按内容抑制噪声	主要记忆风险
Full Attention	是，完整注意力	全历史键值缓存，随长度增长	否	键值状态无限增长
Longformer	是，固定窗口	正负词元 + 最新词元键值	否	窗口外事实不可直接访问
Memformer	是，分段循环状态	固定数量摘要槽位	否（当前基线）	压缩造成细节衰减
Linformer	是，低秩键值投影	投影后的序列表示	否	高秩局部信息丢失
Performer	是，随机特征核	前缀统计量/近似状态	否	近似误差与覆盖问题
Reformer	是，LSH 候选	哈希候选连接	否	碰撞漏检，候选不完整
Keyformer	否，仅推理缓存	重要历史键值 + 最新键值	部分（按重要性）	选择错误后历史不可恢复
本文 Value-token Gated	是，局部注意力外接事实记忆	8 个内容门控语义/词汇槽位	是，写入 + 保留 + 合并	依赖事实边界与门控优化

独立的语言建模筛选中考察稀疏混合专家（mixture of experts, MoE）与该记忆路径的组合，但不把这项筛选与颜色任务的主结果混合。

2 六种基础架构与 SWA 介绍

为避免将不同协议下的数值误读为同一排行榜，本节先给出独立的背景筛选。第一层是在统一 TinyStories/TinyLM 设置下训练 Full Attention、Longformer、Memformer、Linformer、Performer 和 Reformer 六种路径（Beltagy et al. 2020; Wu et al. 2020; Wang et al. 2020; Choromanski et al. 2021; Kitaev et al. 2020），比较语言模型困惑度和系统开销；第二层是在同一骨干上进行 Keyformer 推理期缓存压缩；颜色任务中的选择性事实记忆结果留到后文单独报告。前两层不含随机颜色映射、事实槽位或词级对齐监督。表中的“不适用”表示相应指标在该协议下没有定义，并不表示性能为零。

训练型架构与 Keyformer 的机制概述。 Longformer(Beltagy et al. 2020) 在训练阶段采用滑动窗口注意力，只让窗口内的词元参与交互，以固定状态规模控制计算量。该路径能够限制计算和显存开销，但窗口外的关键事实无法直接访问，存在长期信息丢失问题。

Memformer(Wu et al. 2020) 在训练阶段将输入分成多个片段，把每个片段的历史信息压缩到固定数量的记忆槽，并通过循环状态跨片段传递信息。该架构有直接的跨片段通路，能够降低训练显存开销，但固定槽位存在容量瓶颈，需要额外参数，推理调度也更复杂。

Linformer(Wang et al. 2020) 在训练阶段对键和值序列做低秩投影，使投影维度不再随输入长度变化，从而降低注意力计算开销。该方法依赖低秩表征假设，处理高秩特征时容易丢失信息，投影操作也会带来额外代价。

Performer(Choromanski et al. 2021) 在训练阶段用 FAVOR+ 随机特征近似 softmax 注意力核，避免构建 $N \times N$ 的注意力矩阵，理论上把注意力复杂度降为线性。该近似存在固有误差，性能对随机特征数量敏感，特征变换也会带来额外计算。

Reformer(Kitaev et al. 2020) 在训练阶段利用局部敏感哈希 (LSH) 对词元分桶，只对同桶词元计算注意力，并用因果约束防止时序泄露。哈希碰撞可能丢失有效关联，分桶排序也会造成不规则内存访问，通常只有在超长序列上才可能体现收益。

Keyformer 只作用于推理阶段，不修改训练权重：它根据词元重要性筛选历史键值缓存，同时保留近期窗口内的全部词元。该方法无需重新训练主干，可以和不同注意力架构组合；但筛选和重排会增加推理延迟，过度压缩会丢失远距离信息，也不能降低训练开销。

SWA-sink。 SWA-sink 指 Microsoft 发布于 2026 年 8 月 28 日的“Sliding-window beats linear attention”。该文将四个汇聚词元与固定滑动窗口结合，报告了无需后训练的推理路径。在其多模型下游恢复率实验中，SWA（窗口 64、汇聚词元 4）的 MMLU 恢复率为 $93.2 \pm 3.5\%$ ，六项任务平均恢复率为 $99.0 \pm 0.5\%$ ；作为对照，LoLCATs、Liger-GLA 和 QLinAtt 的平均恢复率分别为 $97.5 \pm 1.3\%$ 、 $92.0 \pm 2.8\%$ 和 $92.9 \pm 0.0\%$ (Jolicoeur-Martineau et al. 2026)。该文的 SWA 结果没有新增训练词元或后训练阶段，因此训练改造开销为零；但它仍然只保留局部词元键值，不能保证跨越窗口的任意事实可被逐字恢复。本文以 SWA-sink 作为基线，在解码器的键值缓存中维护“固定窗口加少量汇聚词元”的局部通道，同时额外维护内容驱动的事实槽位。“Sliding-window beats linear attention”截至本实验记录日期没有可确认的作者代码仓库。因此，本文将 Gemma 的 LOCAL_SLIDING 作为代码级窗口构造参考。

Google Gemma-PyTorch 中的 LOCAL_SLIDING 作为局部滑动窗口的代码参考，但并未在上述论文表中作为独立基准方法报告，因此相应数值记为未测量。本文的 4 个汇聚词元加 124 个最新词元与该类局部窗口设计具有相同的状态约束思想，但额外添加了内容门控事实槽位。

3 六种基础架构实验与结果

本节结果是独立的 TinyStories 语言建模筛选，用于提供注意力路径的背景参照；它们不使用颜色任务的随机姓名-颜色映射、事实槽位或对齐监督，因此不与第 6 的端到端结果组成同一排行榜。

六种训练型架构的稠密前馈层比较。 表 3 是本节的跨双栏总表，汇总统一 TinyStories 语言建模协议下的稠密前馈层结果。训练吞吐包含周期性探针和检查点保存；为便于比较，训练时间由 10^7 个预测词元除以该吞吐得到，只是运行级墙钟时间的近似值。各架构虽然共享 TinyLM 的主要维度和优化设置，但底层实现细节并不完全相同，故表中结论应理解为机制层面的观察，而非严格等条件的因果比较。

在该统一筛选中，Memformer 的 PPL 最低，但它比公共主干多约 11.83% 参数；Longformer 的质量接近 Memformer，却承担了更高的峰值显存；Linformer 和 Performer 具有较高训练吞吐，但当前低秩或随机特征近似会带来质量损失。Full Attention 的吞吐最高主要得益于成熟的 SDPA 路径，不能简单将这一工程优势等同于更低的理论复杂度。Reformer 的吞吐接近 Full Attention，但其质量低于前述三种路径，并且 LSH 候选构造的因果性和稀疏计算核实现更加敏感。上述结果说明，减少理论注意力复杂度并不自动转化为端到端速度优势；训练吞吐还受到张量布局、Python 调度、排序和投影开销的影响。

MoE 对六种骨干的影响。 在全部六层前馈层中加入四专家、每词元选择概率最高的两个专家的无丢弃路由后，统一随机种子 17 的筛选结果如表 4 所示。该设置增加了总参数容量，但每个词元只经过两个专家，因此实际激活参数仍接近稠密前馈层。表中时间同样按一千万预测词元的工作流吞吐换算，不能与专用融合分组矩阵乘法的理论峰值混同。

相对于同一随机种子的稠密版本，MoE 对 Memformer 的 PPL 改善最大（约 8.30%），对 Longformer 和 Lin-

Table 2: 外部滑动窗口与线性注意力文献的结果清单。数值来自 (Jolicoeur-Martineau et al. 2026) 的恢复率表, 不能与本文 TinyLM 的困惑度、完全匹配率或训练时间直接合并; “-” 表示原文未报告。

方法	后训练词元	阶段数	MMLU 恢复率 (%)	六任务平均恢复率 (%)
SUPRA	100B	1	53.0±0.0	88.1±0.0
Hedgehog	40M	2	36.9±0.0	73.9±0.0
LoLCATs	40M	2	83.2±2.2	97.5±1.3
Liger-GLA	20M	1	62.2±5.8	92.0±2.8
MOHAWK	3–5B	3	56.9±0.0	92.4±0.0
Mamba in the Llama	20B	2	67.7±0.0	86.7±0.0
DiJiang	40B	1	88.7±0.0	-
ARWKV	60M/830M	2/3	84.1±0.0	94.7±0.0
Llamba	8–12B	3	91.5±0.0	98.6±0.0
QLinAtt	350–700M	3	74.0±0.0	92.9±0.0
QRWKV6	350–700M	3	92.4±2.7	99.1±0.8
QRWKV7	350–700M	3	86.4±6.8	96.1±4.1
SWA($w = 64, s = 4$)	0	0	93.2±3.5	99.0±0.5

Table 3: 六种基础架构的稠密前馈层结果。时间为一千万预测词元的近似墙钟时间; Keyformer 不属于训练型架构, 故单列于后文。

架构	主要状态/压缩方式	验证 PPL ↓	吞吐 (词元/s) ↑	约训练时间 (min) ↓	峰值显存	参数量	长期事实路径
Full Attention	完整因果键值缓存	18.95 ± 0.12	53,770 ± 585	3.10	2.58 GiB	29.93M	无
Longformer	固定左侧窗口 $W = 128$	17.91 ± 0.08	13,499 ± 269	12.35	6.80 GiB	29.93M	无
Memformer	64 个循环记忆槽位	17.27 ± 0.08	14,670 ± 301	11.36	2.63 GiB	33.47M	固定摘要
Linformer	键值序列低秩投影	18.80 ± 0.11	27,858 ± 106	5.98	2.58 GiB	29.93M	无
Performer	FAVOR+ 随机特征	31.52 ± 0.26	27,079 ± 541	6.15	2.70 GiB	29.93M	无
Reformer	因果 LSH 候选	20.58 ± 0.16	46,939 ± 954	3.55	2.57 GiB	29.04M	无

former 只有约 1.43% 和 0.84%, 对 Performer 几乎没有改善。这一现象不应被解释为 MoE 普遍增强了记忆力: Memformer 同时保留了额外的循环记忆参数, 且所有 MoE 结果只有一个随机种子。更稳妥的解释是, 条件前馈容量与不同注意力状态路径存在交互, 但注意力近似误差或长期信息缺失不能仅靠扩大前馈容量解决。当前普通 PyTorch 调度使训练吞吐下降约 7%–27%、峰值显存增加约 6%–15%; 这反映原型实现而非融合 MoE 内核的上限。

Keyformer 的推理期比较。 Keyformer 改变的是已训练模型的键值缓存, 而不是训练网络。稠密全注意力检查点上, FullKV、Keyformer-75 和 Keyformer-50 的 PPL 分别为 14.91、14.91 和 14.89, 平均键值缓存字节数从 4,709,376 降至 3,538,944 和 2,359,296。在当前实现中, Keyformer-50 的解码吞吐由 219.8 降至 184.2 词元/s, 下降约 16.2%。在 Full Attention+MoE 检查点上也观察到一致方向: 键值缓存字节减少 50%, PPL 基本不变, 但吞吐下降约 17.2%。因此, Keyformer 的主要收益是显存预算, 而非当前 Python 选择与索引收集路径下的速度; 它不应与六种训练型架构共享一个训练时间排名。

4 本文框架介绍

局部键值缓存与长期事实槽位

模型以六层仅解码器 TinyLM 为骨干, 同时维护局部词元缓存和长期事实槽位。局部通道保留 4 个汇聚词元与 124 个近期键值, 承担精确的短期依赖; 长期通道包含 $M = 8$ 个用户事实槽位, 以压缩表示保存跨轮信息。两

类状态共同受固定预算约束, 因此在持续流式推理中不会随历史长度线性增长。

整个状态流转遵循固定的因果顺序: 输入到达后, 事实抽取器定位候选片段, 写入门决定是否保存, 保留门决定已有槽位是否继续占用容量; 下一轮查询触发读取器, 选中的槽位经融合层注入解码器, 最终答案仍由普通语言模型输出头逐词生成。当前轮发现的事实先放入待提交缓冲区, 只有本轮输出完成后才写入可读状态, 因此不会被同一轮提前读取。

在时刻 t , 完整对话状态记为

$$S_t = \{\mathcal{K}_t^{1:N}, \mathcal{V}_t^{1:N}, \mathcal{M}_t^{\text{user}}, \mathcal{M}_t^{\text{asst}}, \mathcal{P}_t, \mathcal{R}_t, \tau_t\}. \quad (3)$$

其中 $\mathcal{K}_t^{1:N}, \mathcal{V}_t^{1:N}$ 是每层的汇聚词元和近期词元键值; $\mathcal{M}_t^{\text{user}}$ 是用户事实槽位; $\mathcal{M}_t^{\text{asst}}$ 是四个固定角色的助手摘要槽位; $\mathcal{P}_t$ 是尚未提交的候选缓冲; $\mathcal{R}_t$ 缓存当前轮选中的检索索引; τ_t 记录绝对位置和轮次边界。所有张量都支持批次重排和按行重置, 使每条会话能够在批处理中独立移动。

若忽略小型元数据, 完整状态的规模约为

$$\mathcal{O}(NBD) + \mathcal{O}(MD + MP), \quad (4)$$

其中 N 是层数, B 是局部键值预算, M 是事实槽位数, P 是每槽位保存的最大事实词元序列长度。本文固定 $N = 6, B = 128, M = 8, P = 12$, 所以生成历史持续增长时, 状态上界保持不变。这里的固定指张量规模上界固定, 并不意味着所有槽位都对每个词元参与计算: 事实读取只在轮次边界发生, 融合层只接收得分最高的 K 个结果。

Table 4: 六种骨干在全层 MoE 下的随机种子 17 结果。Keyformer 使用全注意力加 MoE 检查点, 但本身不参与训练。

架构	验证 PPL ↓	吞吐 (词元/s) ↑	约训练时间 (min) ↓	峰值显存	总参数	每词元激活参数
Memformer	15.7831	12,201	13.66	2.96 GiB	54.71M	33.48M
Longformer	17.6290	12,265	13.59	7.23 GiB	51.17M	29.93M
Full Attention	18.3174	26,504	6.29	2.91 GiB	51.17M	29.93M
Linformer	18.6405	20,340	8.20	2.91 GiB	51.17M	29.93M
Reformer	19.7071	26,821	6.22	2.90 GiB	50.28M	29.05M
Performer	31.7477	20,359	8.19	3.12 GiB	51.17M	29.93M

模型以“用户输入及其后完整助手回复”为一轮。新一轮查询开始时, 读取器只执行一次事实检索, 选中的槽位在本轮生成期间保持不变。当前轮的输出概率计算完成后, 候选事实才被提交到状态, 因此新写入的信息只能影响后续轮次。这种提交顺序保证了因果性, 也使整轮训练与分块流式推理具有一致的状态语义。

具体生命周期包含五个步骤。首先, 用户或系统输入打开新一轮, 读取器在最后一个提示词元处定位相关事实; 该位置的输出概率用于预测第一个助手词元, 因此长期记忆可以影响回答开头。其次, 抽取器在用户词元中定位候选事实片段, 并将其写入有界候选缓冲, 但此时不改变可读槽位。第三, 助手生成过程中复用同一组检索结果, 避免词元间路由抖动, 并通过运行时和计数器累计本轮助手摘要。第四, 到达提交边界时, 模型先计算并输出本轮最后一个词元, 再根据写入门、保留门和淘汰规则更新事实槽位及助手槽位。最后, 轮次关闭, 下一轮才可以读取新提交的事实。

长期事实槽位: 事实抽取与语义槽位

对用户轮中的候选事实片段 S_i , 抽取器首先对片段内隐藏状态做聚合, 并生成语义键和值:

$$\bar{h}_i = \frac{1}{|S_i|} \sum_{j \in S_i} h_j, \quad k_i = W_k \bar{h}_i, \quad v_i = W_v \bar{h}_i. \quad (5)$$

键 k_i 用于内容寻址, 值 v_i 保存关系和事实整体语义。槽位还保存长度受限的事实词元序列、年龄、访问次数、置信度和来源等元数据, 便于审计读写行为并为保留控制器提供依据。每个事实最多保存 12 个词元, 状态上限由槽位数和序列长度上限共同确定。

候选起点由用户角色词元上的起点预测头评分, 长度预测头决定从起点向后复制的词元数。候选数量和事实词元序列长度都预先设定上限, 因此抽取器不会把整段用户消息无界复制到长期状态。训练数据提供事实起点和长度标签; 临时事实仍具有与目标相同的“人物-关系-颜色”句式, 只是其重要性标签不同。这一构造迫使写入门在形式相近的候选之间作选择。

写入、合并与保留门控

写入控制器依据候选表示及上下文预测写入概率

$$g_i^w = \sigma(f_w(\bar{h}_i, m_t)), \quad (6)$$

其中 m_t 表示当前记忆状态的汇总信息。训练阶段采用带直通估计的软阈值, 使离散写入决定仍可接受监督; 评估阶段按阈值执行确定性写入。通过写入门筛选的候选首先与已有键比较: 若最高相似度超过合并阈值, 则由更新门插值新旧表示; 否则写入空槽位。当没有空槽位时, 控制器根据保留分数选择淘汰对象。

对于被寻址到的槽位 r , 更新门根据旧值、候选值与键相似度产生 $g_{i,r}^{\text{upd}}$, 并执行

$$v_r' = (1 - g_{i,r}^{\text{upd}})v_r + g_{i,r}^{\text{upd}}v_i. \quad (7)$$

键和词汇值表示使用相同的门控插值, 离散事实词元序列则在确定性写入发生时替换。若候选与现有事实不够相似且槽位已满, Gated Memory 条件淘汰保留分数最低的槽位; Fixed-LRU 条件使用最后访问时间最早的槽位, 并把写入概率与保留概率固定为 1, 从而形成不依赖随机控制头的确定性基线。

对已激活槽位 r , 保留控制器预测

$$g_r^{\text{ret}} = \sigma(f_{\text{ret}}(k_r, v_r, \text{meta}_r)), \quad (8)$$

低于阈值的槽位在下次提交边界被释放。写入门回答“新事实是否值得进入长期状态”, 保留门回答“旧事实是否仍值得占用槽位”; 二者共同把记忆容量分配转化为内容相关决策, 而不再仅由到达时间决定。

保留特征由槽位键、值以及活跃强度、置信度、年龄、访问次数和冲突标记组成。年龄和访问次数经过缩放后输入线性预测头。这里的冲突标记只表示新旧事实词元序列在重叠位置出现差异, 是一种保守的冲突诊断, 并不是完整的实体属性矛盾解析器。因此, 当前架构具备合并与覆盖接口, 但实验并未声称已经解决开放域知识更新。

稀疏读取与门控融合

查询表示 q_t 与所有活跃事实键计算语义相似度, 并只读取得分最高的 $K = 4$ 个槽位:

$$\mathcal{I}_t = \text{TopK}_r(q_t^\top k_r), \quad \tilde{m}_t = \sum_{r \in \mathcal{I}_t} \alpha_r v_r, \quad (9)$$

其中 α_r 是在候选集合内归一化的注意力权重。检索结果通过共享记忆融合模块注入第 3 和第 6 个解码层。对层 ℓ , 融合可写为

$$h_t^{(\ell)} \leftarrow h_t^{(\ell)} + g_t^{(\ell)} F_\ell(h_t^{(\ell)}, \tilde{m}_t), \quad (10)$$

其中 $g_t^{(\ell)}$ 控制长期记忆对当前词元表示的影响。该设计将“选中哪条事实”和“注入多少记忆信号”分开, 使读取路由与生成融合可以独立诊断。

读取器的查询向量来自新一轮提示边界, 事实键在写入时已归一化。读取得分还结合槽位活跃强度, 使失活槽位无法参与排序。选出前 K 个槽位后, 访问时间和访问次数被更新, 为后续保留和最近最少使用审计提供状态。由于本实验最多每条样本只有一条需要长期保存的目标事实, Recall@1 是最直接的语义寻址指标; Recall@4 则检查目标是否至少进入实际送往融合层的候选集合。

Table 5: 局部键值缓存与长期事实槽位

组件	保存内容	实验上界
局部键值缓存	精确短期词元表示	每层 $4 + 124$
用户事实槽位	跨轮语义事实和值	8 槽
助手槽位	固定角色摘要	4 槽
事实词元序列	可审计的原始词元	12/槽
轮内检索缓存	得分最高的 K 个槽位索引	$K = 4$
候选缓冲	尚未提交的候选	1/轮

值片段与词元对齐路径

未加入词元对齐的 Gated Memory 以完整事实的平均表示作为语义值，并把完整事实词元序列的词嵌入再次平均后参与融合。这种表示适合检索，却弱化了答案对象的边界、顺序和离散身份。Value-token Gated 在保留原语义路径的同时，为事实中的目标值 $V_i \subset S_i$ 增加起点和长度监督：

$$\hat{s}_i = \text{softmax}(W_s H_{S_i}), \quad \hat{l}_i = \text{softmax}(W_l H_{S_i}). \quad (11)$$

模型只聚合值片段内部的隐藏状态，得到独立的词汇值表示

$$z_i = W_{\text{lex}} \left(\frac{1}{|V_i|} \sum_{j \in V_i} h_j \right). \quad (12)$$

槽位新增词汇值表示、值词元编号和有效位置掩码；融合输入由原语义值扩展为 $v_i + z_i$ 。完整事实表示继续承担关系理解和语义检索，值表示则保留精确回答所需的局部词汇信息。

为使 z_i 可直接解码为单词或多词元值，我们为答案位置 p 加入可学习位置嵌入，并使用与语言模型词嵌入共享的词表投影：

$$o_{i,p} = E \text{ LN}(W_o(z_i + e_p)),$$

$$\mathcal{L}_{\text{tok}} = - \sum_p \log P(y_{i,p} | z_i, p). \quad (13)$$

该损失同时施加在写入轮的候选词汇值和查询前实际取出的槽位表示上。前者直接训练值抽取器，后者训练真实记忆状态到答案词元的映射。跨轮状态仍然截断梯度，因此候选级监督也为早期编码器提供必要梯度。该辅助头仅在训练时提供约束；推理时答案仍由原有自回归解码器末端的共享词嵌入输出头生成，模型没有使用复制指针机制。

联合训练目标

设答案语言模型损失为 $\mathcal{L}_{\text{LM}}$ ，事实候选的起点、长度和写入监督分别为 $\mathcal{L}_{\text{start}}$ 、 $\mathcal{L}_{\text{len}}$ 和 $\mathcal{L}_{\text{write}}$ ，语义键对齐、读取和保留损失分别为 $\mathcal{L}_{\text{key}}$ 、 $\mathcal{L}_{\text{read}}$ 和 $\mathcal{L}_{\text{ret}}$ 。完整 Value-token Gated 的训练目标为

$$\begin{aligned} \mathcal{L} = & \mathcal{L}_{\text{LM}} + 0.75\mathcal{L}_{\text{start}} + 0.25\mathcal{L}_{\text{len}} + 0.75\mathcal{L}_{\text{write}} + \mathcal{L}_{\text{read}} \\ & + 0.50\mathcal{L}_{\text{key}} + 0.10\mathcal{L}_{\text{ret}} + 0.01\mathcal{L}_{\text{budget}} \\ & + 0.50\mathcal{L}_{\text{v-start}} + 0.25\mathcal{L}_{\text{v-len}} \\ & + 0.50\mathcal{L}_{\text{cand-tok}} + 0.50\mathcal{L}_{\text{retr-tok}}. \end{aligned} \quad (14)$$

其中 $\mathcal{L}_{\text{cand-tok}}$ 在写入轮使用标注值片段， $\mathcal{L}_{\text{retr-tok}}$ 在查询前使用槽位内保存的词汇值。 $\mathcal{L}_{\text{budget}}$ 惩罚活跃槽位超过目标预算，避免控制器通过无差别激活换取答案性能。Gated Memory 只移除四个词元对齐项，SWA-task 不执行事实写入和读取损失；Fixed-LRU 则将写入、保留和淘汰改为确定性规则。这样，各条件的差异集中在长期状态策略，而不是隐藏的训练预算。

辅助损失的两个施加位置具有互补作用。候选级损失在事实刚被编码时提供梯度，可以缓解跨轮截断梯度造成的信用分配中断；检索级损失使用真实槽位中的词汇值表示，检查写入、状态复制和读取器之后的表示是否仍可解码。若只在候选上监督，模型可能学会从输入事实直接预测值，却没有学会保护和读取槽位；若只在检索后监督，梯度则可能难以回到写入轮的值编码器。两者同时使用是本次改进的核心训练设计。

MoE 与计算容量

为考察计算容量与记忆机制的关系，我们将各解码块的稠密前馈层替换为 4 个结构相同的专家，每个词元选择概率最高的 2 个专家。若路由器概率为 $p_e(h)$ ，则输出为

$$\text{MoE}(h) = \sum_{e \in \text{Top2}(p(h))} \frac{p_e(h)}{\sum_{j \in \text{Top2}} p_j(h)} F_e(h). \quad (15)$$

当前实现采用无丢弃分发，所有词元都会被所选专家处理，并加入负载均衡损失。MoE 只改变前馈层，局部窗口、门控槽位、读取器和值词元对齐路径均保持不变，因此可以检验额外模型容量是否会改变记忆行为或修复词级生成瓶颈。

实验比较原则

为避免把不同数据、初始化模型和监督口径下的结果混成一个排行榜，本文按待验证的问题组织实验。首先，在共同初始化模型上比较三种共同目标条件：FA-TASK（全注意力任务条件）、SWA-TASK（滑动窗口任务条件）和 Memformer。三者只使用相同的因果语言模型损失与答案损失。随后对 Memformer 的循环状态进行清零、交换和逐段重置，以检验答案是否真正依赖跨段状态。再比较原生事实记忆条件 Fixed-LRU 和 Gated Memory；这两个条件保留各自必要的事实候选、写入、读取和保留监督，因此与三种共同目标条件分开报告。

进一步进行严格配对的对齐消融：在 Gated Memory 上加入值片段和记忆到词元的训练信号得到 Value-token Gated，在 Fixed-LRU 上加入同一组信号得到 Fixed-LRU + alignment。两组配对比较均固定初始化

模型、数据、随机种子、训练预算和普通解码器语言模型输出头，因而可以把变化主要归因于词级对齐路径，而不是记忆容量变化。噪声压力测试只在评估阶段加入噪声，不更新参数；另行测量推理状态和吞吐。六种基础注意力架构及稀疏专家使用独立的 TinyStories 语言建模协议，不与颜色任务数值合并。

此外，为单独评估局部缓存本身的作用，我们冻结该初始化模型，只切换局部缓存而不更新参数。完整历史的词元准确率为 41.9609%；加入 4 个汇聚词元后，窗口预算 64/128/256/512 的准确率分别为 41.7793%/42.0221%/42.0009%/41.9625%。这项结果说明冻结语言模型上的滑动窗口本身不会产生长期事实槽位；它与颜色任务中经过微调的 SWA-task 是两个不同的实验。

5 实验结果

实验设置

所有颜色任务均从同一个随机种子 (17) 的 TinyStories 全注意力初始化模型开始。该模型先在 10M 个 TinyStories 预测词元上以长度 512 的上下文和纯下一词元目标训练 1221 个优化步；模型包含 6 层、隐藏维度 384、8 个注意力头、1536 维前馈层，GPT-2 词表大小为 50,257。其独立 TinyStories 验证集词元准确率为 41.9609%，NLL 为 2.9463 (PPL 19.0348)。FA-task 使用无界因果键值缓存；SWA-task 及所有事实记忆条件使用 4 个汇聚词元和 124 个最新词元，总局部预算为 128。因此，FA-task 并不存在两种不同的“窗口内/窗口外”架构，1/4/16 只是同一全注意力模型上的延迟分桶；只有 SWA 的可见范围会随延迟越过最新词元窗口而改变。

事实记忆条件维护 8 个用户槽位，读取器取前 $K = 4$ 个槽位，每轮至多提出一个写入候选，合并相似度阈值为 0.999。门控条件的训练/评估写入和保留阈值分别为 0.1/0.1 与 0.5/0.5；Fixed-LRU 使用确定性的写入和最近最少使用淘汰。所有任务采用 AdamW, $\beta = (0.9, 0.95)$ ，权重衰减 0.1，梯度裁剪 1.0，物理批量 2、梯度累积 2，训练 2000 个优化步。主干学习率为 3×10^{-5} ；事实记忆条件新增的控制器和预测头使用 3×10^{-4} 。计算使用 RTX 4090 的 CUDA BF16 自动混合精度，模型参数保留 FP32。

除噪声压力测试外，所有训练条件的噪声数固定为 0；压力测试不再训练，而是在评估最终检查点时加入 0/2/4/8 条临时事实。共同目标组 (FA-task、SWA-task、Memformer) 优化 $\mathcal{L}_{\text{causal-LM}} + \mathcal{L}_{\text{answer}}$ ，其中答案损失权重为 1。Memformer 是仅解码器的循环记忆适配器，使用其原生隐状态，不加入事实标签、重建损失、值词元对齐或复制指针头。无对齐的 Gated Memory 和 Fixed-LRU 不使用值词元对齐或重建损失；对齐权重（值起点/值长度/候选词元/检索词元）为 0.50/0.25/0.50/0.50；所有条件最终都由解码器末端的共享词嵌入输出头生成答案。

每个长训练运行均完成 2000 步，并记录逐步指标、最终检查点和配置；共同目标运行另保存最佳检查点，对齐运行按协议关闭中间检查点保存。单元测试覆盖状态重置与重排、填充段、值片段掩码、辅助损失梯度和 Memformer 因果性，所有正式运行均无 NaN 或 OOM。稀疏专家数值属于独立的 TinyStories 语言建模筛选，本节不把它们当作颜色任务的配对条件。

数据集构造

每条样本由一条持久用户事实、若干同句式的临时事实、一段从 TinyStories 缓存抽取的填充文本，以及查询和答案组成。事实关系固定为“人物喜欢某种颜色”，颜色词共 16 类（包括 red、blue、yellow 和 navy）。重要的是，姓名-颜色映射按样本独立随机生成，而不是在整个数据集使用一个全局字典；每次样本重置后，模型必须保存当前样本的具体映射。

训练时持久事实使用 Remember/Keep/Save 提示，临时事实使用 Temporary/Skip/Ignore 提示；验证时分别改用未见的 Store 和 Discard。每个分段长度为 32 个词元，延迟分桶为 1, 4, 16；最长填充文本至少含有 16×32 个干扰词元，连同轮边界远超 SWA 的 124 个最新位置。主验证集只使用噪声数 = 0，每个延迟取 24 个留出样本，共 72 条。噪声压力测试才使用噪声数 = 0, 2, 4, 8 与三个延迟的笛卡尔积，每个单元仍为 24 条，共 288 条；两种样本规模不混用。

对比条件与评价指标

颜色任务包含七个条件，并沿用以下正式名称：共同目标组为 FA-task、SWA-task 和 Memformer；原生事实记忆组为 Fixed-LRU、Gated Memory、Value-token Gated 以及 Fixed-LRU + alignment。共同目标组共享普通因果语言模型损失和答案损失；原生事实记忆组保留事实候选、写入、读取和保留监督。因此，表格提供同一数据协议下的数值，但不把不同监督口径伪装成单一公平排名。Value-token Gated 是 Gated Memory 加完整值片段和记忆到词元对齐，Fixed-LRU + alignment 是其严格配对的固定规则版本。Memformer 不加入重建或对齐。

下文表格中的“噪声数”表示临时事实数量，“延迟段数”表示填充分段数量。端到端质量以贪心生成的完全匹配率 (EM) 和教师强制答案 NLL 衡量。记忆行为使用目标事实存活率、Recall@K、MRR、平均活跃槽位数和噪声接收率。值路径还记录值片段起点、长度、片段完全匹配，以及记忆词元的逐词元准确率和序列完全匹配率。SWA-task 不包含长期槽位，因此其存活率和召回率记为“不适用”，而不是零。

本文对这些指标作如下定义。若目标槽位在回答前仍处于激活状态，则计为一次存活成功；若其在读取器排序中位于前 K ，则计为 Recall@K 成功。对单目标任务，目标排名为 r 时倒数排名为 $1/r$ ，所有样本的平均值构成 MRR。EM 要求完整生成序列（包括答案后的结束符）与标准序列完全一致；生成另一个颜色词、遗漏多词元片段或在答案后继续生成都计为错误。噪声接收率是被硬写入的临时事实候选数除以临时候选总数，反映控制器是否把容量让给不应长期保存的信息。

教师强制 NLL 只在标准答案位置计算，用于区分“模型已将正确词放在高概率位置”和“自由生成真正完成答案”这两个层面。值片段完全匹配要求起点和长度同时正确；记忆词元序列完全匹配要求词汇值预测的全部有效位置逐一正确。对于多词元值，这些指标比首词元准确率更严格，也能发现模型只预测值的前缀却没有正确终止的情况。

共同目标与原生记忆条件的主结果

表 6 给出七个条件在噪声数 = 0、共 72 个留出样本上的最终结果。延迟列对应三个独立的、每格 24 条样本的测

试单元。共同目标组 (FA-task、SWA-task、Memformer) 使用完全相同的因果语言模型损失和答案损失；事实记忆组含有各自必要的结构监督 (Gated Memory 使用事实起点/长度、写入、键对齐、读取、保留和预算项；Fixed-LRU 使用确定性写入/淘汰及相应读取项)，故表中数值用于定位机制差异，而不是把七个条件压成单一统计排名。

共同目标对照显示，在事实仍位于局部窗口的短延迟条件下，FA-task 和 SWA-task 都能学习随机姓名-颜色复制；随着延迟增加，FA-task 的本次训练泛化也下降，而 SWA-task 在延迟 4/16 时还受到局部可见范围的限制。Memformer 在相同主任务损失下三个延迟均为 100%，不能归因于额外的事实标签或重建损失。原生事实记忆条件的 Fixed-LRU 和 Gated Memory 端到端结果分别为 70.83% 和 65.28%；它们的检索指标需结合下一表解读。

在噪声数为 0 时，两种无对齐的记忆条件都能保存并读到目标事实，却不能稳定把连续的词汇值交给普通语言模型输出头，说明“检索成功”不等于“答案成功”。对齐消融的配对差值为：Value-token Gated 相对 Gated Memory 的 EM 提高 34.72 个百分点 (答案 NLL 0.5961 $\rightarrow$ 0.000408)，Fixed-LRU + alignment 相对 Fixed-LRU 提高 29.17 个百分点 (0.4113 $\rightarrow$ 0.000276)。两组的目标存活、读取命中和活跃槽位均不变，因而主要增益发生在检索值到离散词元的利用，而不是容量或读取排序。

长延迟与噪声压力测试

噪声压力测试仅在评估阶段进行，不更新模型参数，只改变临时事实数量。表 8 报告三个延迟测试单元的 EM 平均，因此噪声数为 0 的列与主表一致，其他列不是新的训练结果。固定的随机数据生成器会使某些分桶出现非单调波动 (例如 SWA-task 的噪声数为 4 高于噪声数为 2)，所以这里报告完整分桶而不拟合单调曲线。

噪声为 8 时，Fixed-LRU 的目标存活/读取命中 @4 平均为 93.06%/0%，而 Gated Memory 为 100%/52.78%；这说明噪声首先造成槽位竞争和读取失败。Memformer 在噪声为 0 的正常状态下仍为 100%，但加入两条临时事实后迅速退化，表明其隐状态对填充文本扰动较敏感。由于这些模型没有使用噪声训练，不能把这些数字解释成经过抗噪训练后的鲁棒性。

对两个对齐检查点进行相同的只评估压力测试。这里同时报告 EM 和读取命中 @4，便于区分“值片段仍可重建”和“目标槽位仍被读到”：

两个对齐条件在所有噪声分桶的值片段完全匹配率都为 100%。Value-token Gated 的记忆词元完全匹配率也保持 100%，所以高噪声下的损失主要来自目标存活、读取和融合，而非词汇值本身。相反，Fixed-LRU + alignment 在噪声为 8 时目标存活平均只有 40.28%，记忆词元完全匹配率随之下降；固定替换策略的淘汰是其主要瓶颈。

参数、状态与运行开销

推理基准在 RTX 4090、BF16、批量大小 = 1、噪声数 = 0、延迟段数 = 16 的 523 个词元前缀上测量前缀填充和“查询 + 贪心解码”。表 10 报告的是推理基准，不是训练吞吐；状态字节数是处理完前缀后的持久状态逻辑大小。当前实现按最大容量预分配，不能直接等同于 CUDA 分配器显存。

FA-task 的完整历史键值缓存随前缀增长，SWA-task 的局部状态有界；Memformer 的隐状态最小，但当前 Python 循环调度使前缀填充最慢。事实记忆状态同时包含语义、词汇和元数据张量，因而比仅含隐状态的 Memformer 更大。Value-token Gated 中的对齐路径只增加约 0.305M 参数，不改变槽位状态字节数。该吞吐反映研究原型的 Python/PyTorch 实现，不能外推到融合内核或生产级循环内核；独立 TinyStories 筛选的资源结果也不与此表混写。

Memformer 状态的因果性

在同一 Memformer 最终检查点上进行状态干预，不更新任何参数。表 11 显示，正常传递前缀状态时三个延迟均为 100%；清零、交换批内状态或每个分段后重置都会使结果降至接近随机基线。这排除了模型只依赖查询模板或位置计数的解释。

将状态缩放到 0.5 倍时 EM 仍为 62.50/58.33/50.00% (延迟 1/4/16)，缩放到 1.0 倍恢复满分；在本检查点上加入小幅高斯状态噪声也没有改变 EM，说明必要性主要由状态是否存在决定，而不是由微小数值扰动决定。该实现是仅解码器的 Memformer 风格适配器，并非完整的编码器-解码器 MemBART 复现。

训练充分性与可复现性

所有长训练均完整运行 2000 步，无 NaN/OOM。训练曲线显示共同目标函数从约 17 降至后期的 1-2 区间，Memformer 的留出答案词元准确率在末步达到 100%；事实监督项和主语言模型损失也持续下降。两个对齐运行的最终总目标函数约为 0.000931 (Value-token Gated) 和 0.000641 (Fixed-LRU + alignment)，训练期间最佳值分别为 0.0001659 和 0.0001482。训练曲线中的准确率是候选记忆词元准确率，不是普通答案 EM，二者不能互换解释。

对齐运行没有保存 step=500/1000/1500 的完整检查点，因此本文不再虚构检查点回扫表；最终留出集 EM/NLL 和逐步指标是正式报告依据。训练曲线证明优化在预算内稳定，但单一随机种子仍不能替代跨初始化的重复实验。

Table 6: 随机种子 17、噪声数为 0 的颜色任务主结果（每个延迟 24 条、共 72 条）。EM 为自由贪心生成的完全匹配率；答案 NLL 只在标准答案位置计算；首词元排名越低越好。不同监督组不作单一公平排名。

方法	延迟 =1 EM (%)	延迟 =4 EM (%)	延迟 =16 EM (%)	总体 EM (%)	答案 NLL	首词元排名
FA-task	95.83	87.50	4.17	62.50	0.6791	3.708
SWA-task	79.17	0.00	4.17	27.78	1.4591	34.833
Memformer	100.00	100.00	100.00	100.00	0.0170	1.000
Fixed-LRU	95.83	58.33	58.33	70.83	0.4113	1.542
Gated Memory	95.83	54.17	45.83	65.28	0.5961	1.903
Value-token Gated	100.00	100.00	100.00	100.00	0.000408	1.000
Fixed-LRU + alignment	100.00	100.00	100.00	100.00	0.000276	1.000

Table 7: 原生事实记忆条件的写入与读取诊断（噪声数为 0）。表中“读 @4”为读取命中率，“值段”和“记忆词元”分别为值片段与记忆词元完全匹配率；“-”表示未启用对齐头。

方法	EM (%)	存活 (%)	读 @4 (%)	槽位	值段 (%)	记忆词元 (%)
Fixed-LRU	70.83	100.00	100.00	1.00	-	-
Gated Memory	65.28	100.00	100.00	1.00	-	-
Value-token Gated	100.00	100.00	100.00	1.00	100.00	100.00
Fixed-LRU + alignment	100.00	100.00	100.00	1.00	100.00	100.00

Table 8: 只评估噪声压力测试的 EM（%，每个噪声分桶含三个延迟、共 72 条）。

方法	噪声 =0	噪声 =2	噪声 =4	噪声 =8
FA-task	62.50	13.89	9.72	2.78
SWA-task	27.78	5.56	19.44	6.94
Memformer	100.00	5.56	2.78	1.39
Fixed-LRU	70.83	22.22	9.72	8.33
Gated Memory	65.28	8.33	13.89	5.56

Table 9: 对齐模型噪声压力测试。单元格为 EM / 读取命中 @4（%，三个延迟的平均）。

方法	噪声 =0	噪声 =2	噪声 =4	噪声 =8
Value-token Gated	100.00/100.00	43.06/100.00	15.28/68.06	16.67/48.61
Fixed-LRU + alignment	100.00/100.00	20.83/100.00	5.56/62.50	11.11/6.94

Table 10: 推理效率与状态大小（随机种子 17，BF16，批量大小 =1，延迟段数 =16）。前缀填充/解码为词元每秒；峰值为分配器 MiB。

方法	参数量 (M)	前缀状态 (B)	前缀填充 (词元/s)	解码 (词元/s)	峰值 (MiB)
FA-task	31.855	4,885,106	19,674	189.2	250.9
SWA-task	31.855	1,186,376	12,522	139.8	247.4
Memformer	33.476	294,928	2,376	177.5	256.1
Fixed-LRU	31.855	1,242,516	8,444	87.8	250.4
Gated Memory	31.855	1,242,514	8,094	84.7	251.0
Value-token Gated	32.160	1,242,514	7,976	83.5	252.4
Fixed-LRU + alignment	32.160	1,242,516	8,148	84.4	251.9

Table 11: Memformer 循环状态因果干预（EM 为%，答案 NLL 为三个延迟的总体值）。

状态处理	延迟 =1	延迟 =4	延迟 =16	答案 NLL
正常状态	100.00	100.00	100.00	0.018
清零记忆	8.33	8.33	8.33	2.845
交换记忆	0.00	0.00	0.00	3.729
逐段重置	8.33	8.33	8.33	2.845

6 结果分析

选择性记忆的收益来自容量分配

共同目标对照把可见范围和状态机制分开：事实仍在局部窗口内时，FA-task 与 SWA-task 都能学习随机映射；延迟为 4/16 时，SWA-task 明显受到最新词元缓存的限制。FA-task 在延迟为 16 时的下降来自本次训练预算和泛化不足，不能错误地写成全注意力的窗口上限。Memformer 在相同的语言模型加答案损失下三个延迟均为 100%，说明显式的跨段状态通路足以保存样本特定映射。

进一步表明，固定写入和内容门控都能在噪声数为 0 时保持目标存活与读取命中 @4 为 100%，但 EM 只有 70.83% 和 65.28%。因此，存活/读取回答“事实是否还在、读取器是否找到它”，EM 才回答“生成器是否把它转成正确字符串”；外部记忆模型不能只报告其中一个环节。

语义检索与精确生成之间存在表示鸿沟

Gated Memory 的读取命中率为 100% 而 EM 仅为 65.28%，是“检索正确而生成失败”的直接证据。完整事实的语义池化适合相似度寻址，却没有明确要求连续状态与某个离散答案词元对齐；普通共享词嵌入输出头因而可能在多个颜色词之间选错。对齐消融保持相同槽位、读取器和噪声数为 0 的协议，仅加入值片段、词汇值及候选/检索记忆词元损失，就使 Value-token Gated 的 EM 提高 34.72 个百分点；在 Fixed-LRU 上的配对提高 29.17 个百分点。两组存活和读取命中不变，支持增益主要来自词汇值到词表的直接训练信号，而非“记忆写得更多”。

该对齐实验仍是组合消融：值片段、词汇融合、候选级损失和检索后损失同时启用，所以不能把 34.72 或 29.17 个百分点拆分归因到单一子模块。对齐头只提供训练和诊断信号，最终答案始终由普通解码器语言模型输出头生成；这也避免把辅助头的准确率误报成自由生成 EM。

MoE 与外部记忆解决不同问题

稀疏专家结果来自独立的 TinyStories 筛选，并没有在颜色任务协议中重新训练，因而不应被放进当前主表。机制上，稀疏专家增加的是每层前馈网络的条件容量，外部事实槽位增加的是跨轮状态；前者不能自动恢复已经从局部键值缓存中删除的随机映射，也不能替代记忆到词元的对齐。该区分是为什么本文同时报告独立架构筛选和当前配对对齐，而不把两者的绝对 EM 直接比较。

综合推理基准，系统存在明确的状态-速度权衡：FA-task 的完整键值状态最大但前缀填充最快，SWA-task 状态有界，Memformer 的隐状态最小却受 Python 循环调度影响，事实记忆状态则因语义、词汇和元数据而更大。上述数字是当前实现的可复现实测，而不是与硬件无关的复杂度定理。

7 结束语

不足之处

首先，全部正式结果仅来自随机种子 17。主表的 100% 是单次运行在 72 条噪声数为 0 的验证样本上的点估计；噪声压力测试的 288 条只属于评估阶段，不能被当作主

实验样本量。当前尚不能报告跨随机种子的均值、标准差、置信区间或统计显著性。

其次，压力测试仍是封闭的合成任务。答案集合只有 16 个颜色值，关系固定为“人物喜欢颜色”，提示模板数量有限。尽管验证使用了未见提示词、较长延迟和更高噪声，模型仍可能学习颜色分类与固定句法映射；结果不能外推为开放域实体、数字、代码或知识更新。

第三，值边界来自生成器的人工监督。真实对话没有天然的值偏移，模型需要自动标注、弱监督或关系抽取来发现应精确恢复的子片段。对齐头不是复制指针头；面对未登录词、长数字串或罕见多词元实体，普通共享词嵌入输出头仍可能失败。

第四，本文已提供 Gated Memory 与 Fixed-LRU 的严格配对消融，但仍不能分离值片段、词汇融合、候选级损失和检索后损失各自的贡献。对齐收益是组合路径的因果证据，不是每个预测头的独立效应估计。

第五，当前实现仍是研究原型。槽位张量按最大容量预分配，逻辑激活槽位不会直接减少物理显存；滑动注意力和循环/稀疏专家调度没有生产级融合内核。推理只测试贪心解码，尚未比较采样或束搜索；结果主要来自最终检查点，而非跨随机种子的提前停止选择。

未来展望

近期工作应先补充其他随机种子，在完全相同协议下报告多种子均值和不确定性；随后做最小对齐消融（单独移除值片段、候选级损失、检索后损失或词汇融合），并扫描槽位容量、Top-K、写入/保留阈值和弱化提示词，确认控制器学习的是内容而不是少数提示词。

任务层面应扩展日期、整数、小数、长数字、罕见多词元字符串和随机标识符，并加入多关系、查询改写、否定、冲突更新与同一实体多属性；容量压力也应超过八个槽位。只有在更开放的值类型上仍出现“存活/读取高而 EM 低”时，才应引入复制指针或词元级交叉注意力，并同时量化准确率、状态和延迟。

工程层面可压缩激活槽位并实现融合的滑动窗口/循环内核，避免把逻辑状态节省误解为物理显存节省；稀疏专家也应在融合的分组矩阵乘法下重新测量。理想分层是：局部键值缓存负责短期逐词上下文，语义槽位负责跨轮选择性保持，词元路径负责原样恢复，MoE 只在任务复杂度确实需要时提供额外条件容量。

参考文献

References

- Iz Beltagy, Matthew E. Peters, and Arman Cohan. 2020. Longformer: The long-document transformer. arXiv preprint arXiv:2004.05150.
- Alexia Jolicoeur-Martineau, Rhea Sanjay Sukthankar, Pashmina Cameron, and Emy Gervais. 2026. Sliding-window beats linear attention. arXiv preprint arXiv:2608.28444.
- Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. In Advances in Neural Information Processing Systems.
- Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. 2024. Efficient streaming language models with attention sinks. In International Conference on Learning Representations.

Qingyu Wu, Zhenzhong Lan, Jingyu Li, and others. 2020. Memformer: A memory-augmented transformer for sequence modeling. arXiv preprint arXiv:2010.06891.

Sinong Wang, Belinda Z. Li, Madian Khabsa, Han Fang, and Hao Ma. 2020. Linformer: Self-attention with linear complexity. arXiv preprint arXiv:2006.04768.

Krzysztof Choromanski, Valerii Likhoshesterov, Dohan Yoon, et al. 2021. Rethinking attention with performers. In International Conference on Learning Representations.

Nikita Kitaev, Łukasz Kaiser, and Anselm Levskaya. 2020. Reformer: The efficient transformer. In International Conference on Learning Representations.